

TD de Programmation Orientée Objet

Licence d'Informatique

Semaine du 6 avril 2026

Mise en œuvre du MVC et du design pattern Command dans le jeu Zork

Le Model View Controller (MVC) est un principe architectural qui repose sur l'idée de séparer le code en trois catégories indépendantes :

Model les classes qui gèrent les données de l'application (cf. figure 1). Aucune de ces classes ne doit interagir directement ou indirectement avec l'utilisateur (i.e. aucun affichage), ces classes ne dépendent pas non plus des commandes que l'utilisateur aura la possibilité d'effectuer. Ces classes sont complètement indépendantes des classes du Controller et de la View.

Controller les classes qui exécutent les commandes de l'utilisateur comme : se déplacer, prendre ou déposer un objet dans une pièce, afficher les objets présents dans une pièce ou portés par le joueur. Ces classes font appel aux classes de la catégorie View pour connaître les demandes de l'utilisateur et aux classes du Model pour exécuter les commandes. C'est parmi les classes de cette catégorie que l'on va mettre en œuvre le design pattern *Command*. Comme pour les classes du Model, ces classes ne doivent pas interagir directement avec l'utilisateur, cependant ce sont les classes qui déclenchent les interactions avec l'utilisateur en faisant appel aux classes de la View (cf. figure 2).

View toutes les classes qui interagissent directement avec l'utilisateur (cf. figure 3). Ces classes ne dépendent pas des classes des catégories Model ou Controller.

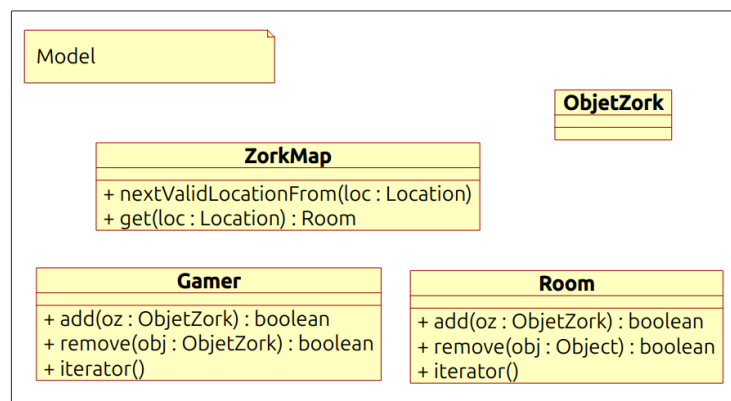


FIGURE 1 – Les classes du Model

Exercice 1 Définition des classes RoomInventory et GamerInventory.

Les classes RoomInventory et GamerInventory implémentent toutes les deux l'interface Command. La méthode void execute() de la classe RoomInventory affiche la liste des ObjetZork présents dans la pièce courante, puis affiche le menu principal de l'application à l'aide de la méthode showMenu(). Pour cet affichage des ObjetZork on utilise une instance d'une classe implémentant l'interface ListView<E> dont la méthode show() affiche une liste numérotée d'objets de type E. Pour obtenir une instance d'une classe implémentant l'interface ListView<E> on utilise la méthode :

```
public static <T> ListView<T> buildMenu(Iterable<T> items)
```

De manière semblable à la classe `RoomInventory`, la classe `GamerInventory` affiche la liste des `ObjetZork` transportés par le joueur. On supposera que les classes `Room` et `Gamer` implémentent l'interface `Iterable`.

Définissez les classes `RoomInventory` et `GamerInventory`.

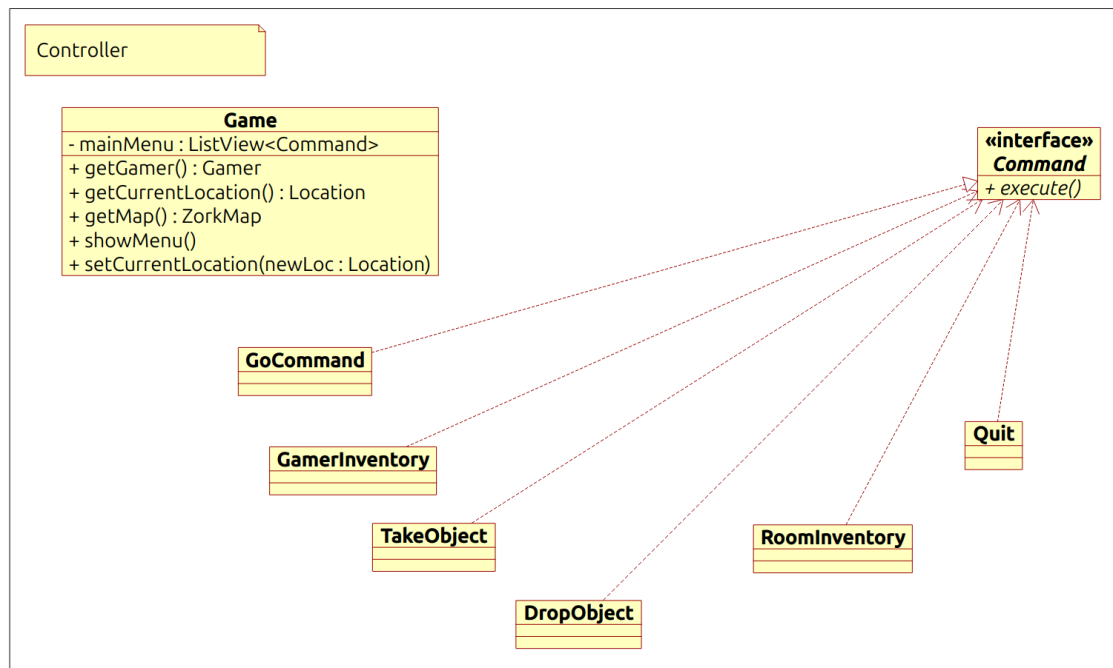


FIGURE 2 – Les classes du Controller

Exercice 2 Définition des classes `DropObject` et `TakeObject`.

La classe `DropObject` implémente la commande permettant au joueur de déposer un objet dans la pièce courante et la classe `TakeObject` implémente la commande permettant au joueur de prendre un objet dans la pièce courante. Pour choisir l'objet à prendre ou à déposer on utilisera à nouveau `ListView` avec sa méthode `show()` pour afficher la liste des `ObjetZork` et sa méthode `readChoice()` pour récupérer l'`ObjetZork` choisit par le l'utilisateur.

Définissez les classes `DropObject` et `TakeObject`.

Exercice 3 Définition de la commande `GoCommand`.

`GoCommand` est la commande qui permet au joueur de se déplacer vers une pièce voisine de la pièce courante. Pour cette commande on demandera à l'utilisateur de faire un choix entre les directions dans lesquelles se trouve une pièce voisine. Pour obtenir ces directions on utilisera la méthode suivante de la classe `ZorkMap` :

```
public EnumMap<Direction, Room> nextValidLocation(Location loc)
```

ainsi que la méthode `Set<K> keySet()` de l'interface `Map` qui renvoie un `Set` contenant toutes les clés d'une `Map`.

Définissez la classe `GoCommand`.

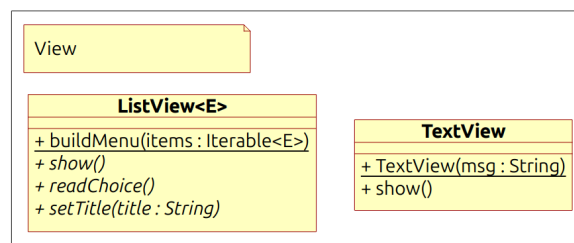


FIGURE 3 – Les classes de la View

Exercice 4 Implémenter la méthode `showMenu()` de la classe `Game`. On supposera que le joueur gagne dès qu'il récupère l'objet `TRESOR`.